Report
NLU Assignment-2

-By Pari Sharma (B22CS039)

Problem-1(Link)

TASK-1: DATASET PREPARATION

Data Collection

For this assignment, I collected text data from multiple IIT Jodhpur sources:

- Main IIT Jodhpur website
- Department websites (mainly **CSE, Electrical, Bio, Chemical, Mathematics**)
- Faculty profile pages (around **5–6 faculty members per department**)
- Academic regulation and course curriculum PDFs

The data was stored in structured folders:

- iitj_pages/ → main website + department pages
- faculty_pages/ → faculty information
- pdf_data/ → academic regulations and curriculum

Data Cleaning (Preprocessing)

The collected data was not clean initially, so I performed the following steps:

- Removed URLs
- Removed numbers
- Removed extra symbols and punctuation
- Converted all text to lowercase
- Tokenized the text into words

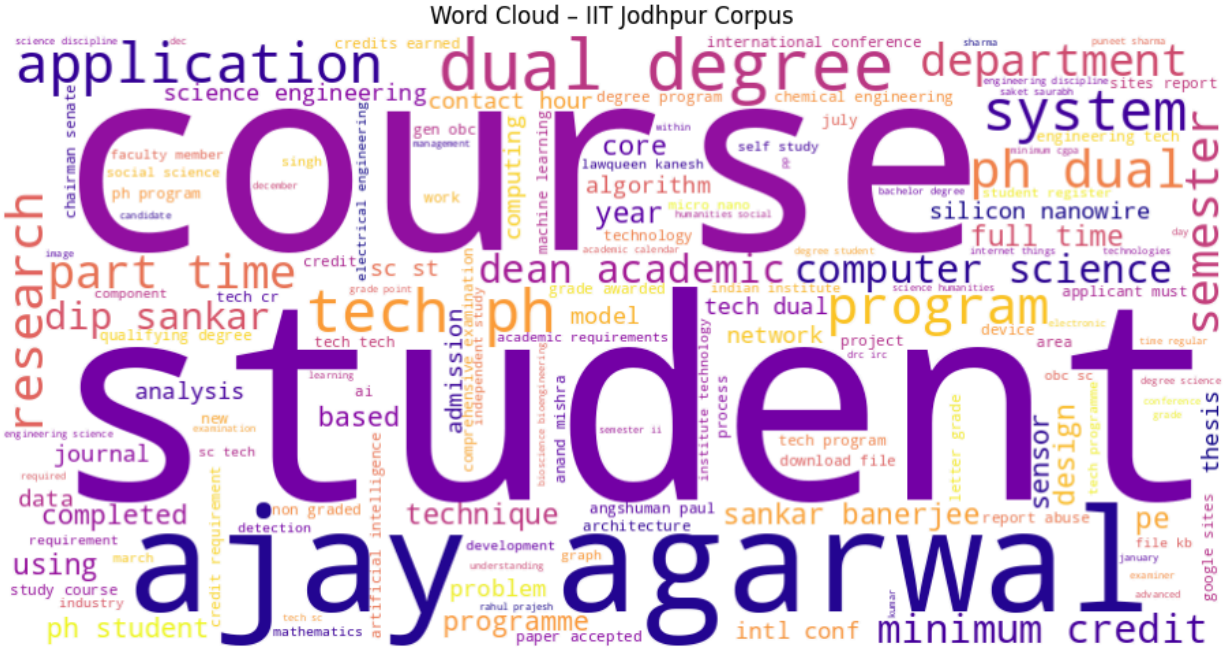
After cleaning, we get a proper text corpus that can be used for training.

Dataset Output

```
Total documents : 35
Total tokens    : 60,527
Vocabulary size : 6,882
Total sentences : 7,663
```

Word Cloud

Word Cloud – IIT Jodhpur Corpus



The word cloud represents the most frequent words in the dataset. Words like *course*, *student*, *program*, *research*, *dual degree*, and *phd* appear much larger compared to others. This is because most of the collected data comes from academic sources such as department websites, curriculum documents, and academic regulations.

TASK-2: MODEL TRAINING

In this task, CBOW and Skip-gram (SGNS) models were trained using different hyperparameters such as embedding size (50, 100), window size (2, 4), and negative samples (5, 10).

Glimpse of Hyperparameter Tuning:

```
--------------------------------------------------
 CBOW  |  dim=50  win=2  neg=5  ep=5
--------------------------------------------------
  Epoch 1/5  loss=4.1588  lr=0.02500
  Epoch 2/5  loss=4.1529  lr=0.02083
  Epoch 3/5  loss=4.0325  lr=0.01667
  Epoch 4/5  loss=3.6793  lr=0.01250
  Epoch 5/5  loss=3.3906  lr=0.00833
 CBOW training complete.

==================================================
 SGNS  |  dim=50  win=2  neg=5  ep=5
==================================================
  Epoch 1/5  loss=3.9742  lr=0.02500
  Epoch 2/5  loss=2.9108  lr=0.02083
  Epoch 3/5  loss=2.4128  lr=0.01667
  Epoch 4/5  loss=2.1837  lr=0.01250
  Epoch 5/5  loss=2.0599  lr=0.00833
Skip-gram training complete.

--------------------------------------------------
 CBOW  |  dim=100  win=4  neg=5  ep=5
--------------------------------------------------
  Epoch 1/5  loss=4.1588  lr=0.02500
  Epoch 2/5  loss=4.1561  lr=0.02083
  Epoch 3/5  loss=4.0821  lr=0.01667
  Epoch 4/5  loss=3.7657  lr=0.01250
  Epoch 5/5  loss=3.4554  lr=0.00833
 CBOW training complete.

==================================================
 SGNS  |  dim=100  win=4  neg=5  ep=5
==================================================
  Epoch 1/5  loss=3.5302  lr=0.02500
  Epoch 2/5  loss=2.3704  lr=0.02083
  Epoch 3/5  loss=2.0541  lr=0.01667
  Epoch 4/5  loss=1.8986  lr=0.01250
  Epoch 5/5  loss=1.8072  lr=0.00833
Skip-gram training complete.
```

- For CBOW, the loss decreased gradually across epochs (e.g., ~4.15 → ~3.39). Increasing embedding size gave slight improvement. Using more negative samples increased the initial loss but improved learning over time.
- For Skip-gram, the loss decreased faster and reached lower values (e.g., ~3.97 → ~2.05, and up to ~1.80 for higher dimensions). It consistently performed better than CBOW across all settings.

Overall, Skip-gram gives better results, while CBOW is simpler and faster.

TASK-3: SEMANTIC ANALYSIS

Nearest Neighbors (Cosine Similarity)

1. **Word: Research**

- CBOW: learning, india, using, high
- Skip-gram: proposal, representative, excellence, interests, members

Observation:
CBOW gives very general words which are not strongly related to research. Skip-gram gives more relevant terms like proposal and excellence, which are clearly connected to research activities. Hence, Skip-gram captures better semantic meaning.

2. **Word: Student**

- CBOW: registration, academic, completed, semester, students
- Skip-gram: falls, disciplinary, visiting, attendance, appear

Observation:
CBOW captures formal academic terms, which are useful but basic. Skip-gram provides contextual words like attendance and disciplinary, showing better real-world understanding.

3. **Word: Phd**

- CBOW: students, must, programme, year, institute
- Skip-gram: mtech, pursuing, interested, enrolled, admit

Observation:
CBOW outputs generic words, while Skip-gram captures the academic journey (pursuing, enrolled), making it more meaningful.

4. **Word: Exam**

- CBOW: conducted, cgpa, earned, branch, head
- Skip-gram: intimated, clearing, passed, quiz, internship

Observation:
CBOW focuses on formal terms, whereas Skip-gram captures actions related to exams, which better reflects actual usage.

5. **Word: Department**

- CBOW: institute, systems, pe, research, core
- Skip-gram: head, moderation, leading, professor

Observation:
CBOW gives general institutional words, while Skip-gram identifies roles within a department, making it more semantically meaningful.

Analogy Experiments

1. UG : BTech :: PG : ?

- CBOW: mtech, program, enrolled
- Skip-gram: mtech, mba

Remarks:
Both models correctly map PG to MTech, which is semantically correct.

2. professor : research :: student : ?

- CBOW: students, registration, thesis, courses
- Skip-gram: carry, visiting, disciplinary, work

Remarks:
CBOW provides relevant academic terms like thesis, making more sense. Skip-gram is less accurate here.

3. undergraduate : course :: phd : ?

- CBOW: academic, requirements, study
- Skip-gram: engagement, title, practicals

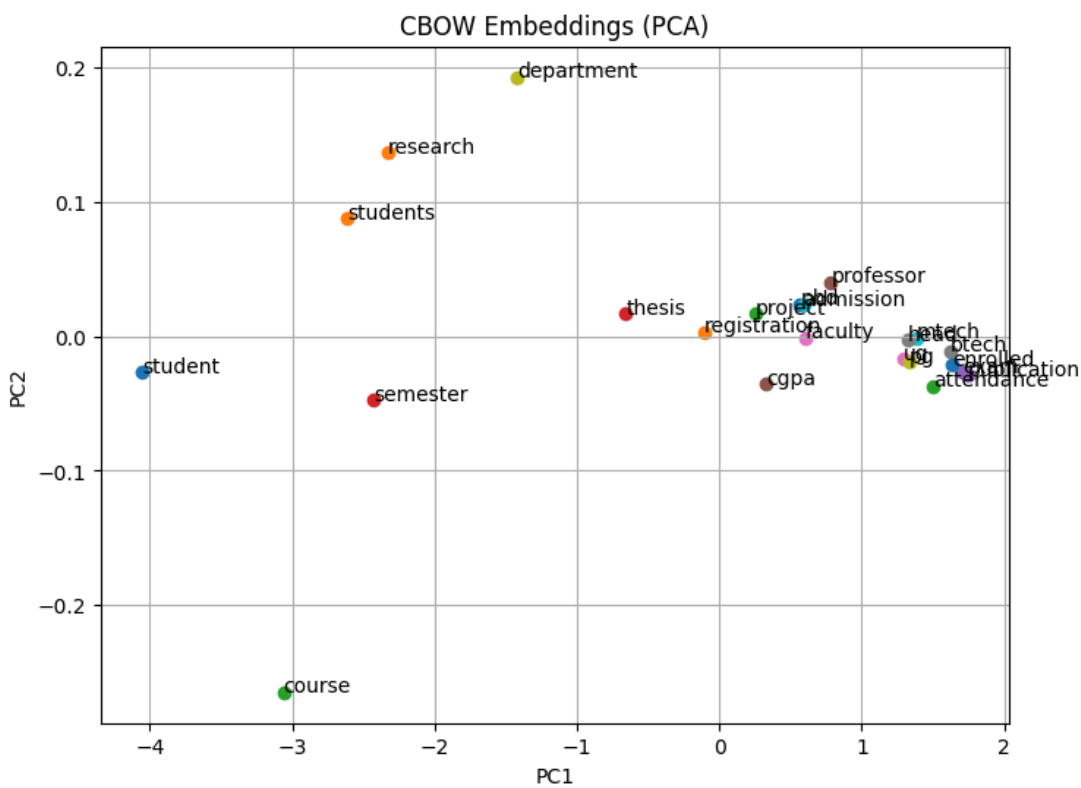
Remraks:
CBOW better represents academic structure, while Skip-gram results are less clear.

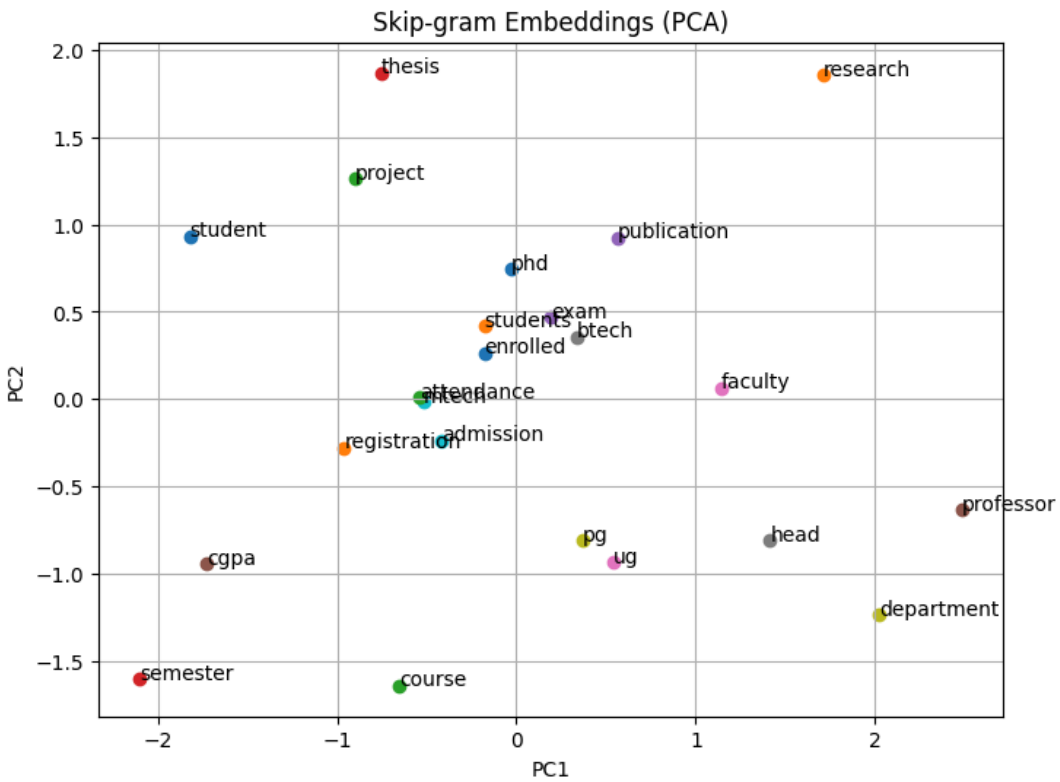
4. bachelor : undergraduate :: master : ?

- CBOW: data, technology, department
- Skip-gram: postgraduate, developing, cse

Remarks:
Skip-gram correctly identifies postgraduate, showing better understanding of degree hierarchy.

TASK-4: VISUALIZATION





PCA was used to project embeddings of words such as *student, students, course, semester, exam, cgpa, ug, btech, pg, mtech, phd, research, project, thesis, publication, professor, faculty, head, department, admission, enrolled, registration, attendance* into 2D space.

- In **CBOW**, some related words like *btech, mtech, ug, pg, exam* are grouped together, but overall the clusters are weak. Important words like *student, course, research* are placed far apart, and many words are scattered, showing limited semantic grouping.

- In **Skip-gram**, the clustering is clearer and more meaningful. Degree-related words (*btech, mtech, phd*) are closer, role-based words (*professor, faculty, head, department*) are also relatively grouped. Action-related words like *registration, admission, attendance* also appear close.

Skip-gram shows better and more meaningful clustering compared to CBOW, indicating stronger semantic understanding of word relationships.

Problem-2(Link)

TASK-0: DATASET

A dataset of 1000 Indian names was generated using a Large Language Model and stored in TrainingNames.txt. The dataset includes a wide variety of names to capture diverse naming patterns.

Preprocessing

- All names were converted to lowercase and a character-level vocabulary was created from all unique characters in the dataset.
- Three special tokens were added:
    - <PAD> : for padding sequences
    - <SOS> : start of sequence
    - <EOS> : end of sequence

Encoding

Each name is converted into a sequence of numbers using a mapping from characters to indices. First, a <SOS> token is added at the beginning of the name, and an <EOS> token is added at the end. This helps the model understand where a name starts and ends.

Batch Preparation

Since names have different lengths, they cannot be directly processed together. To solve this, all sequences in a batch are padded with the <PAD> token so that they have the same length. During training, the loss function ignores these padding tokens, ensuring that they do not affect learning.

TASK-1: MODEL IMPLEMENTATION

Three sequence models were implemented from scratch for character-level name generation:

1. Vanilla Recurrent Neural Network (RNN)
2. Bidirectional Long Short-Term Memory (BLSTM)
3. RNN with Attention Mechanism

These models take character sequences as input and predict the next character at each time step.

Common Hyperparameters

- Embedding Size: 64
- Hidden Size: 256
- Learning Rate: 0.001
- Epochs: 120
- Batch Size: 64
- Optimizer: Adam
- Gradient Clipping: 5.0

1. Vanilla RNN

Architecture Flow:

- Input: sequence of character indices
- Embedding Layer: converts each character into a dense vector
- Recurrent Layer: computes hidden state using current input and previous hidden state
- Output Layer: maps hidden state to vocabulary scores

At each time step, the hidden state is updated using the current input and previous state

Trainable Parameters

- 91,806 parameters

2. Bidirectional LSTM (BLSTM)

Architecture Flow:

- Input: sequence of character indices

- Embedding Layer: converts characters into dense vectors
- Forward LSTM: processes sequence left → right
- Backward LSTM: processes sequence right → left
- Concatenation: combines forward and backward hidden states
- Output Layer: predicts next character

Each LSTM cell contains:

- Forget gate
- Input gate
- Candidate state
- Output gate

These gates control how information is stored, updated, and passed through time.

Trainable Parameters

- 674,718 parameters

3. RNN with Attention

Architecture Flow:

- Input: sequence of character indices
- Embedding Layer: converts characters into vectors
- Recurrent Layer: computes hidden states sequentially
- Attention Mechanism:
    - Computes similarity between current hidden state and all previous hidden states
    - Generates attention weights using softmax
    - Produces a context vector as a weighted sum of past hidden states
- Output Layer: combines current hidden state and context vector to predict output

Trainable Parameters

- 165,022 parameters

TASK-2: QUANTITATIVE EVALUATION

Evaluation Results

Model	Params	Novelty	Diversity
RNN	91,806	0.970	1.000
BLSTM	674,718	0.930	0.920
ATTN	165,022	0.930	1.000

Observations

1. Vanilla RNN

The Vanilla RNN achieves the highest novelty (97%) and perfect diversity (1.0), showing that it generates new and varied names instead of memorizing the dataset. The steady loss decrease indicates stable learning, and despite its simplicity, it captures character-level patterns effectively.

2. BLSTM

The BLSTM shows lower novelty (93%) and diversity (0.92), and often generates short or meaningless outputs (e.g., “aru”, “gge”). This suggests overfitting. Furthermore, because name formation is intrinsically left-to-right, using bidirectional context is not a good fit. The backward pass introduces future information that is not available during generation, leading to confusion and gibberish outputs.

3. RNN with Attention

The Attention model outperforms BLSTM in terms of output quality, achieving high novelty (93%) and perfect variety (1.0). The attention mechanism helps the model focus on relevant past characters, improving structure while maintaining diversity.

TASK-3: QUALITATIVE ANALYSIS

1. Vanilla RNN

Sample Outputs

- yuvanej siddiqui
- nakel sidvin
- bhavanesh kapoor
- aadhila iyer
- manan reddy

Analysis

The Vanilla RNN generates names that look quite *realistic and natural*. Many names follow a common Indian pattern with a first name and a surname, such as *bhavanesh kapoor* and *manan reddy*. Even when the names are new, they still sound believable. This shows that the model has learned how characters usually combine to form names.

Failure Modes

- Small spelling mistakes (e.g., *nakel sidvin*)
- Some slightly unusual names like yuvanej

2. BLSTM

Sample Outputs

- aru, gge, er, bel, demal
- abjey, van, rangos, va, yag
- mir, taty, chavin, chmax

Analysis

In this task, the *BLSTM does not do well*. The majority of outputs don't appear like real names and are either extremely brief or nonsensical. It is unable to produce correct full names.This indicates that the model is probably overfitting even when the training loss is

quite modest. Another issue is that BLSTM uses information from both directions. But in name generation, characters are produced from left to right. So using future information (backward pass) does not make much sense here and leads to confusing or broken outputs.

Failure Modes

- Very short outputs (e.g., *er*, *va*)
- Random or gibberish text (e.g., *gge*, *chmax*)
- No proper name structure

3. RNN with Attention

Sample Outputs

- darshan singh
- jihit gupta
- zaya nair
- deven reddy
- sarika kapoor
- alok iyer
- archana sharma
- ayush kapoor

Analysis

The majority of the names produced by the Attention model are *well-structured and clear.* Many names, including Darshan Singh and Archana Sharma, are very similar to actual Indian names. It outperforms BLSTM in maintaining appropriate structure.The attention mechanism enhances the overall quality by assisting the model in concentrating on significant portions of the name.

Failure Modes (similar to Vanilla-RNN)

- Some spelling errors (e.g., *chatterje*)
- Slightly unusual or mixed names

Important Note

The training dataset is very diverse, covering many types of Indian names from different regions and styles. However, it contains only 1000 names, which is relatively small.

Because of this, the models have to learn a wide variety of patterns from limited data. This makes the task harder and explains why:

- Some names are slightly incorrect
- Some outputs are unusual or mixed